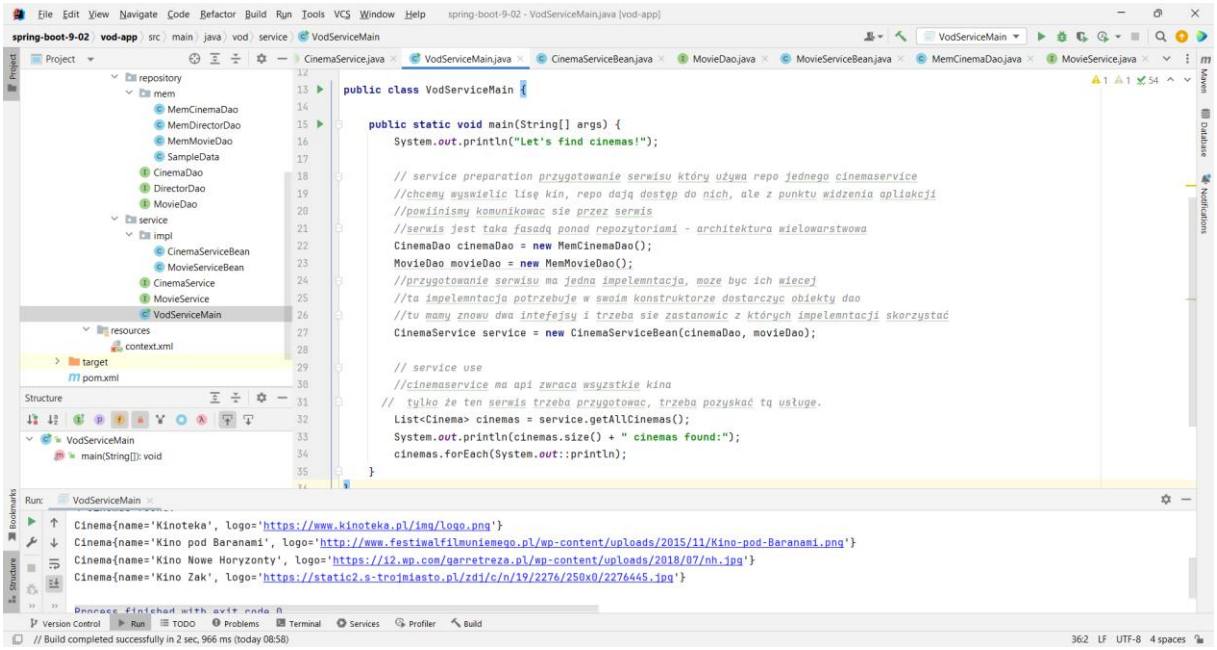
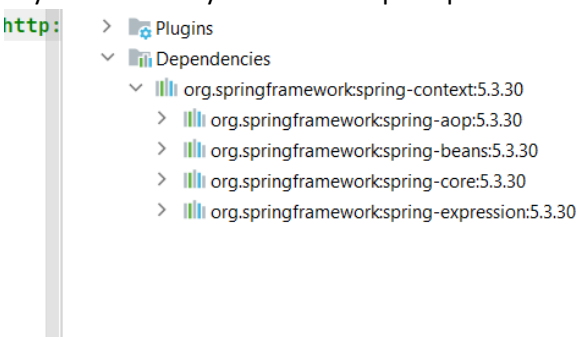
Podstawowe zachowania, mechaniki spring kontenera - **kontekst Springa**



Potrafimy uruchomić klasę główną. Chcemy się **pozbyć przygotowania** serwisu a zostawić tylko użycie. Dodaliśmy zależność do pliku pom.xml – dodany kontekst ściągnął nam inne zależności



Wszystkie są dorzucone na classpath.
Chcemy korzystać z serwisu ale przygotowanie zostawić Springowi.
Obiekt klasy CinemaService chcemy otrzymać z kontekstu springa.
Spring oferuje **kontekst aplikacyjny – interfejs ApplicationContext** który ma wiele implementacji.
Użyjemy takiej, która opiera się na adnotacjach (inne to xml) - Kontekst aplikacyjny konfigurowany adnotacjami (parafrazując nazwę klasy na to co ona sobą reprezentuje).

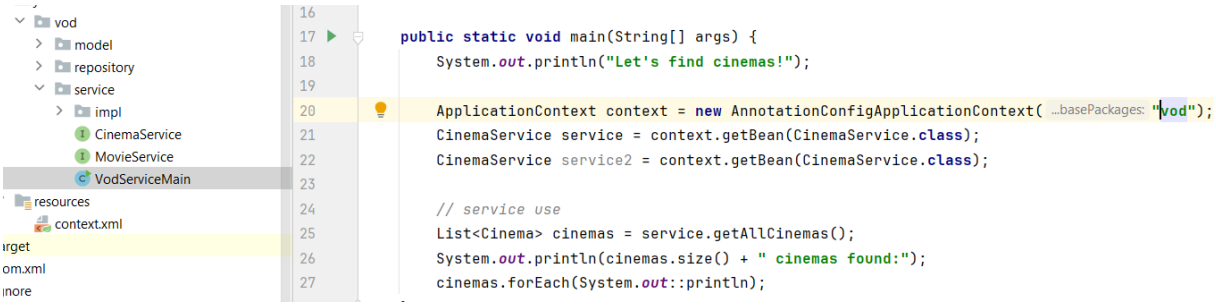
Zamiast:

```
public static void main(String[] args) {
    System.out.println("Let's find cinemas!");
    CinemaDao cinemaDao = new MemCinemaDao();
    MovieDao movieDao = new MemMovieDao();
    CinemaService service = new CinemaServiceBean(cinemaDao, movieDao);
    List<Cinema> cinemas = service.getAllCinemas();
    System.out.println(cinemas.size() + " cinemas found:");
    cinemas.forEach(System.out::println);
```

otrzymamy:

```
public static void main(String[] args) {
    System.out.println("Let's find cinemas!");
    ApplicationContext context = new
AnnotationConfigApplicationContext("vod");
    CinemaService service = context.getBean(CinemaService.class);
    //uzywamy kontekstu do pobrania beana typu interfejsowego CinemaService
    // service use
    List<Cinema> cinemas = service.getAllCinemas();
    System.out.println(cinemas.size() + " cinemas found:");
    cinemas.forEach(System.out::println);
}
```

Ma ona kilka konstruktorów.



Nasz cały kod jest w pakiecie **vod**.
Dlatego w konstruktorze wskaż nazwę pakiet bazowego, który zostanie automatycznie przeskanowany w runtime.
W momencie wywołania klasy będzie przeskanowany cały classpath w poszukiwaniu klas, które są w pakiecie vod zagnieżdżone i wszystkie adnotacje zostaną wychwycone, zinterpretowane – utworzone obiekty zostaną umieszczone w kontekście aplikacyjnym, który jest referencją do kontenera springowego z zawartością różnych beanów.
Musimy spowodować wprowadzenie tych beanów do kontekstu, ale zaczynamy od strony użycia springa – używamy kontekstu do pobrania beana – **getBean** i używamy typu interfejsowego CinemaService.class.
Czyli od strony klienta nie ma tu relacji/zależności od klasy implementującej.
Mamy kontekst i do niego się zwracamy o otrzymanie obiektu zgodnego z typem interfejsowym.
Interfejsy to de facto Typ i Api.
Spring nie działa w trybie zachłannym/eager, czyli że wszystko co znajdzie umieści w kontekście tylko **będzie szukał adnotacji springowych, które teraz dodamy**, aby spowodować żeby CinemaServiceBean był komponentem.

Zaczniemy od warstwy repozytorium – mamy dwa interfejsy w warstwie serwisów i trzy w warstwie repozytoriów.
Każdy z tych serwisów ma po jednej implementacji ale może ich być więcej. Zaczniemy budować zależności w strategii Bottom-up – czyli zaczynamy od repozytoriów.

Dla CinemaDao implementacją jest MemCinemaDao – **Spring nie włącza zachłannie każdej klasy, którą znajdzie do kontekstu tylko te które są opisane albo w xmlu albo w adnotacjach**.
Bazowa to **@Component**, od tego zaczniemy.
Adnotacja markuje klasę – gdy spring skanuje klasy i znajdzie taką z nałożona adnotacją Component to automatycznie utworzy obiekt tej klasy przy pomocy domyślnego konstruktora lub gdy go brak przy użyciu innego.
Tu mamy konstruktor bezparametrowy więc jest to prostsza sytuacja.
Nie widzimy go ale on jest dodawany automatycznie do klasy, gdy żaden inny nie jest zadeklarowany.
Obiekt jest utworzony w kontekście springa, który sobie odnotowuje, że jest to obiekt typu MemCinemaDao, który implementuje interfejs CinemaDao.
Jeżeli ktoś będzie chciał z kontekstu wyciągnąć obiekt typu interfejsowego to otrzyma ten obiekt.
Dodaj adnotacje do kolejnych klas:

```
main
java
vod
  model
  repository
    mem
      MemCinemaDao
      MemDirectorDao
      MemMovieDao
      SampleData
    CinemaDao
    DirectorDao
    MovieDao
  service
    impl
      CinemaServiceBean
```

```
@Component
public class MemMovieDao implements MovieDao {
    1 usage
    @Override
    public List<Movie> findAll() { return SampleData.movies; }

    1 usage
    @Override
    public Movie findById(int id) {
        return SampleData.movies.stream().filter(m -> m.getId() == id).fin
    }

    1 usage
    @Override
    public List<Movie> findByDirector(Director d) {
```

Teraz przejdź warstwę wyżej z warstwy repozytoriów do warstwy serwisów.

```
DirectorDao
MovieDao
service
  impl
    CinemaServiceBean
    MovieServiceBean
CinemaService
MovieService
VodServiceMain
resources
```

Dwa serwisy, które grupują operacje wokół kin i filmów.

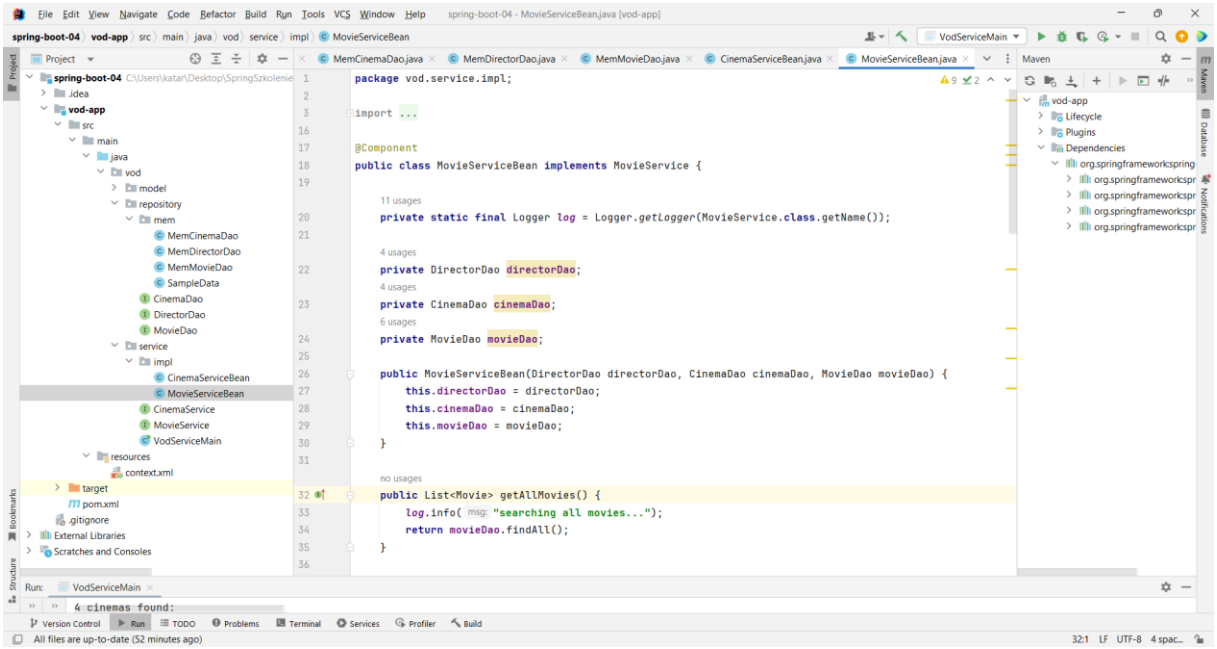
```
import java.util.List;
import java.util.logging.Logger;

@Component
//@Scope("prototype")
public class CinemaServiceBean implements CinemaService {

    5 usages
    private static final Logger log = Logger.getLogger(CinemaService.class.getName());

    4 usages
    private CinemaDao cinemaDao;
    2 usages
    private MovieDao movieDao;

    //@Autowired
    public CinemaServiceBean(CinemaDao cinemaDao, MovieDao movieDao) {
        log.info( msg: "creating cinema service bean");
        this.cinemaDao = cinemaDao;
        this.movieDao = movieDao;
    }

    no usages
    @Override
    public Cinema getCinemaById(int id) {
        log.info( msg: "searching cinema by id " + id);
        return cinemaDao.findById(id);
    }
```

Dodajemy adnotacje aby klasa była umieszczona w kontekście.
Ta klasa ma zależności do dwóch CinemaDao i MovieDao, jest jeden konstruktor i zostanie on użyty, wymaga parametrów, spring bierze je z kontekstu.

Spring najpierw zeskanuje wszystkie komponenty, posprawdza sobie zależności i **zbuduje w pamięci drzewo zależności**, zacznie od liści, później będzie je scalać dostarczając odpowiednie referencje liści do konstruktora węzłów – **automatyczne wiązanie - mechanizm authowired**.

Jeśli jest jeden **konstruktor** to niczego więcej nie potrzebujemy, **gdy jest ich więcej** musimy użyć adnotacji **@Authowired,** która markuje nam dany konstruktor, który ma być zawołany w momencie tworzenia obiektu i nakazuje jego użycie.

```
private MovieDao movieDao;

//@Autowired
public CinemaServiceBean(CinemaDao cinemaDao, MovieDao movieDao) {
    log.info( msg: "creating cinema service bean");
    this.cinemaDao = cinemaDao;
    this.movieDao = movieDao;
}

no usages
```



Dodaj adnotację do drugiego serwisu. Ten serwis przyjmuje trzy dao i każdy zostanie automatycznie dostarczony.
Może być też taka sytuacja, że mielibyśmy konstruktor bezparametrowy a zależności byłyby dostarczane w inny sposób.

Metody Injection: wstrzyknięcie przez konstruktor, przez setter i przez pole.

```
@Autowired
private DirectorDao directorDao;
4 usages
```

Spring do tego pola bezpośrednio przypisze wartość używając **Reflection Api** ingerując w strukturę tego obiektu w czasie wykonania/runtime – uzyska dostęp do prywatnego pola i przypisze wartość.

Uwaga: Jest to odradzana technika ze względu na to, że w pewnych środowiskach wykonawczych modyfikacja modyfikatora private może być zakazana.

Ale kiedyś bardzo szeroko stosowany więc można się spotkać z tą techniką w projektach.

```
11 usages
20  private static final Logger log = Logger.getLogger(MovieService.class.getName());
21
22  @Autowired
23  priv
    4 usag
24  priv
    6 usag
25  priv
26
27  publ
28
29
30      this.movieDao = movieDao;
31  }
32
```

Field injection is not recommended

org.springframework.beans.factory.annotation

```
@Target({ElementType.CONSTRUCTOR,ElementType.METHOD,ElementType.PARAMETER,ElementType.FIELD,ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Autowired
extends java.lang.annotation.Annotation
```

Maven: org.springframework:spring-beans:5.3.30 (spring-beans-5.3.30.jar)

Wygeneruj setter do jednego pola np. DirectorDao.

```
1   package vod.service.impl;
2
3   import ...
16
17  @Component
18  public class Movi
19
    11 usages
20      private stati                                   vice.class.getName());
21
22      //@Autowired
    4 usages
23      private Direc
    4 usages
24      private Cinem
    6 usages
25      private Movie
26
27      public MovieS                                   cinemaDao, MovieDao movieDao) {
28          this.dire
29          this.cine
30          this.movi
31      }
32
    no usages
33      public List<M
34          log.info(
35          return mo
36      }
```

Select Fields to Generate Setters

Template: IntelliJ Default

- vod.service.impl.MovieServiceBean
  - directorDao:DirectorDao
  - cinemaDao:CinemaDao
  - movieDao:MovieDao

OK Cancel

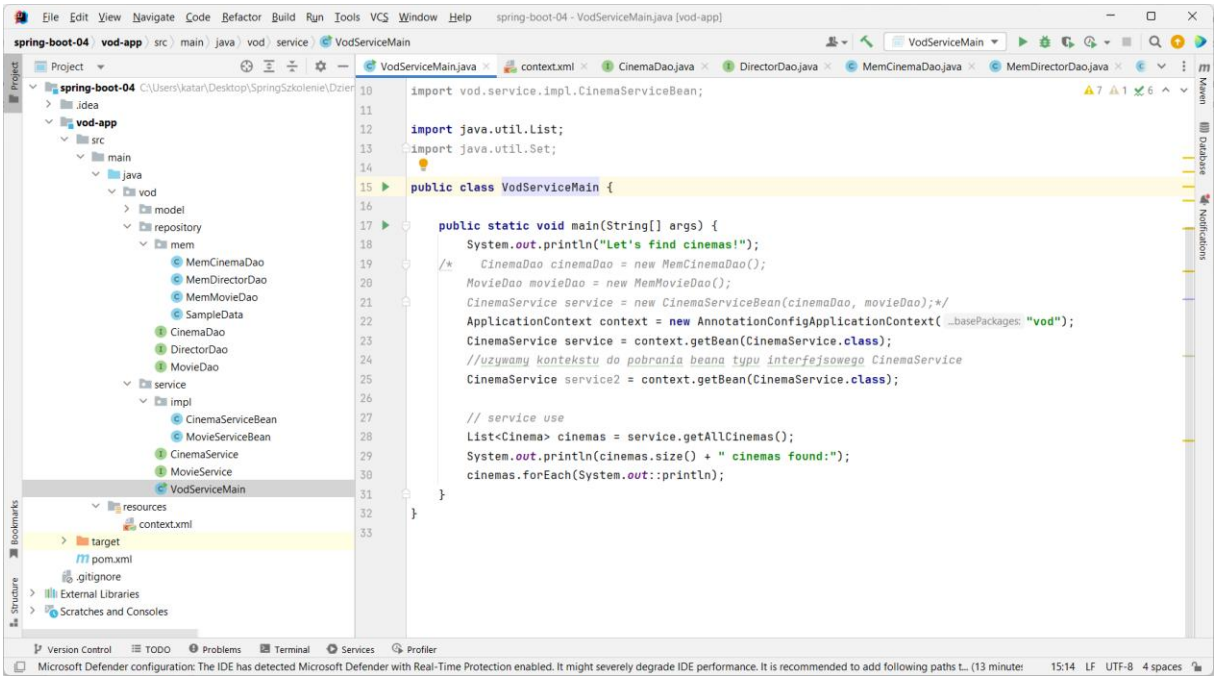
```
        this.cinemaDao = cinemaDao;
        this.movieDao = movieDao;
    }

    @Autowired
    public void setDirectorDao(DirectorDao directorDao) {
        this.directorDao = directorDao;
    }
```

I wtedy adnotacja może się znaleźć na poziomie metody dowolnie nazwanej.

Wówczas **kolejność działań** jest następująca: najpierw utworzony zostanie obiekt przy pomocy konstruktora bezparametrowego, następnie kontekst springa zawoła metodę oznaczoną adnotacją set i dostarczy implementacji.
Trend jest taki aby wstrzykiwać zależności przez konstruktor bo w przypadku ich braku nie zostanie obiekt utworzony wcale, a w dwóch pozostałych strategiach, zostanie, ale później w momencie wstrzykiwania wystąpi błąd procesu tworzenia kontekstu springa – aplikacja się nie podniesie.

Adnotacja przy konstruktorze jest wymagana tylko, gdy mamy więcej konstruktorów, żeby wskazać który ma być użyty.

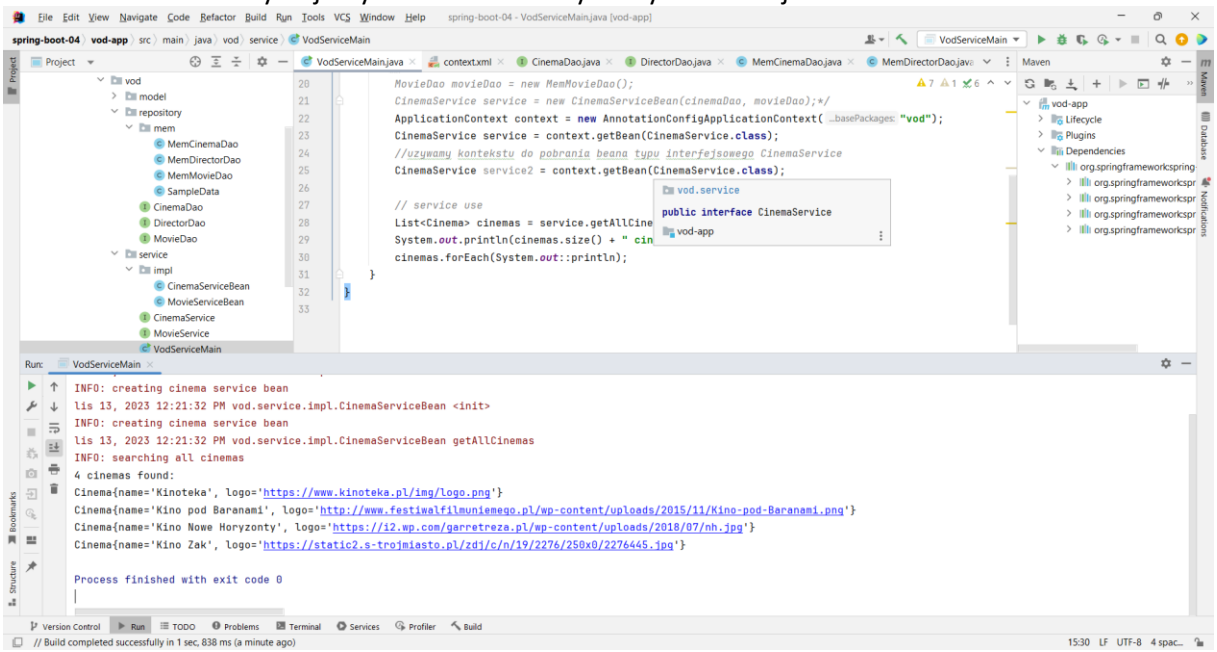
Gdyby było więcej implementacji obiektów Dao to wówczas stosujemy **jedną z technik wyznaczania kwalifikacji,** która implementacja ma się pojawić – adnotacja Primary.
Każda adnotacja ma swój target – czyli miejsce w kodzie gdzie kompilator pozwala na jej umieszczenie.
Reasumując:



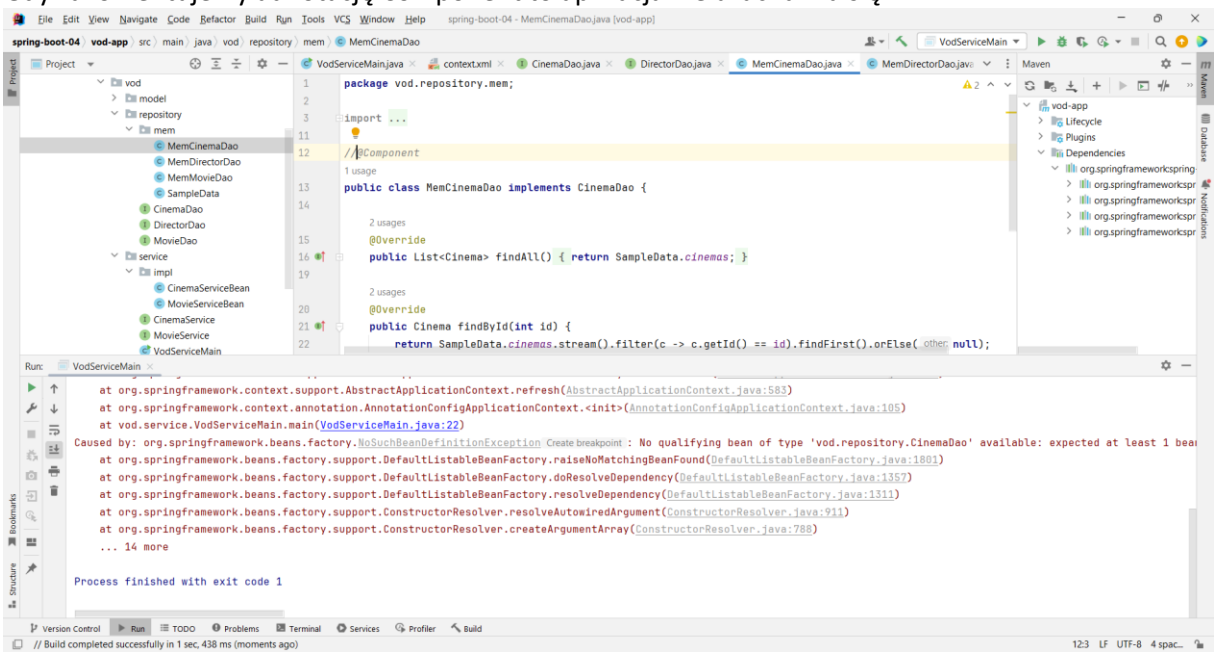
```java
public class VodServiceMain {

    public static void main(String[] args) {
        System.out.println("Let's find cinemas!");

        ApplicationContext context = new AnnotationConfigApplicationContext( ...basePackages: "vod");
        CinemaService service = context.getBean(CinemaService.class);
        //uzywamy kontekstu do pobrania beana typu interfejsowego CinemaService
        CinemaService service2 = context.getBean(CinemaService.class);

        // service use
        List<Cinema> cinemas = service.getAllCinemas();
        System.out.println(cinemas.size() + " cinemas found:");
        cinemas.forEach(System.out::println);
    }
}
```

Klasa AnnotationConfigApplicationContext implementująca interfejs ApplicationContext skanuje pakiet bazowy vod w poszukiwaniu adnotacji springowych i buduje wszystkie beany, u nas 3 klasy Dao z konstruktorem bezparametrowym, później skanuje dalej, model omija bo to są dane, które nie powinny być komponentami, przechodzi do warstwy usług, ma dwa z konstruktorami parametrowymi więc to odnotowuje i wyznacza graf tworzenia kolejności i idzie od tych co nie mają zależności, tworzy obiekty i rejestruje je w kontenerze jako obiekty klas i interfejsów.

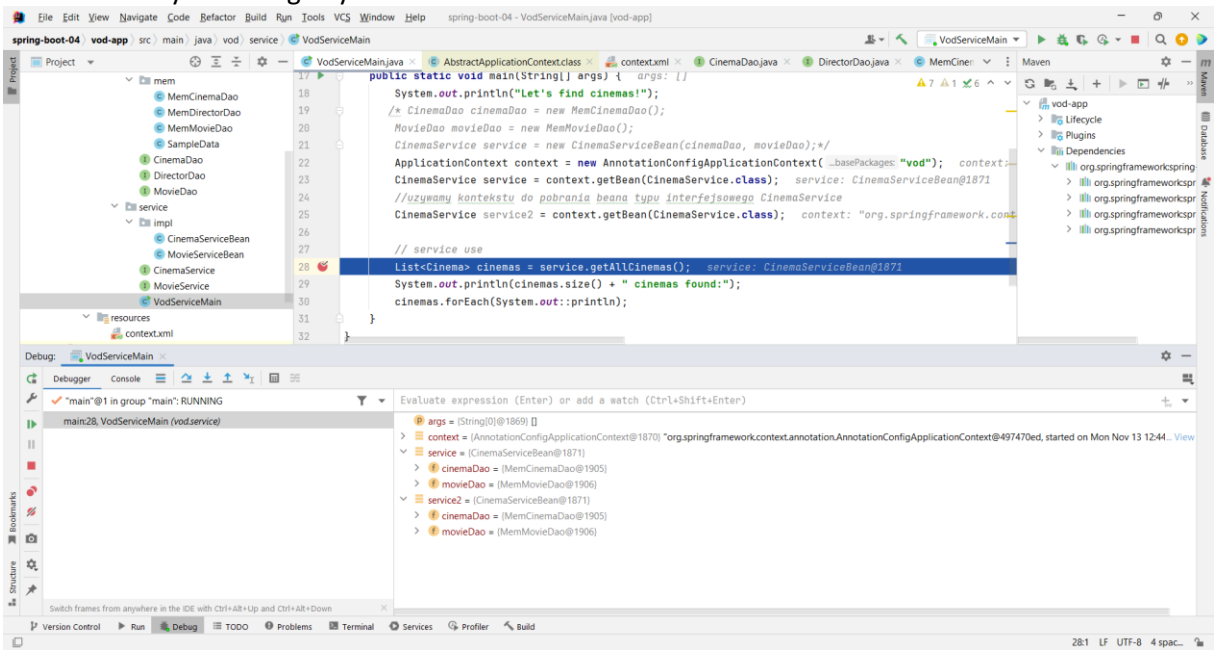
Obiekty rezydują obok siebie w kontenerze springowym.
Konfiguracja następuje automatycznie w momencie bootstrapu aplikacji.
Klient woła skonfigurowany serwis i korzysta z jego api.
Po uruchomieniu otrzymujemy to samo ale korzystamy z adnotacji i kontekstu:



Gdy zakomentujemy adnotację Component to aplikacja nie uruchamia się.



Uruchom w trybie debugowym.

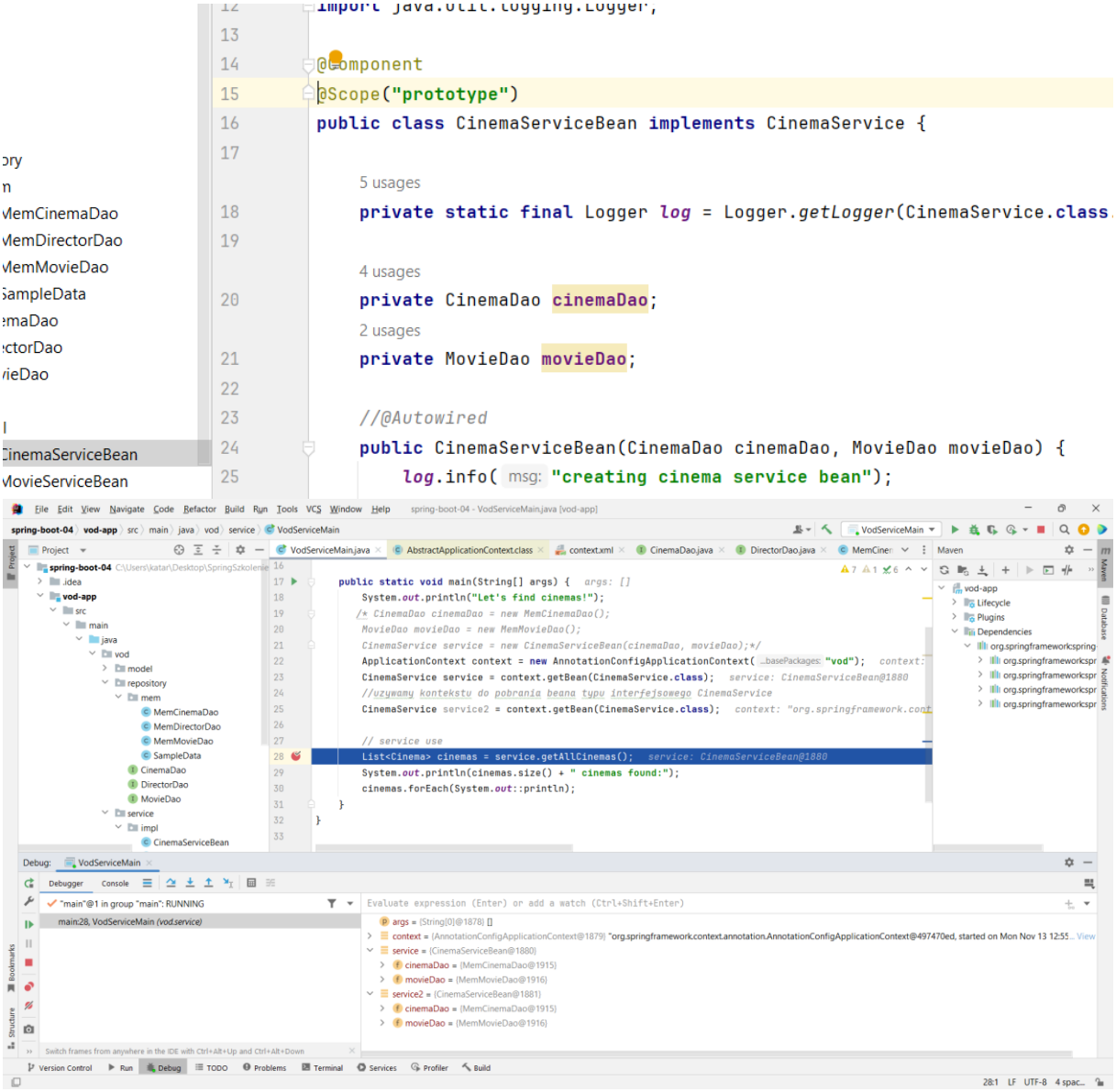


Otrzymujemy serwis i jego implementacje – czyli wymagane referencje zostały tu automatycznie dostarczone.
Od strony klienta nie komponujemy już obiektu serwisu tylko otrzymujemy z kontekstu komponent skonfigurowany. (step over)

Zwróć uwagę na drugi obiekt service2 i wstrzyknięte do niego referencje.
Serwis jest binem ma referencję/numerek – identyfikator obiektu w pamięci Javowej.
Wartość jego nie ma znaczenia ale jest to unikalny identyfikator.
Mamy tu ten sam identyfikator dla obu beanów serwisu oraz wstrzykniętych obiektów dao.

Jest to związane z zakresem – scopem, który domyślnie dla Springa jest singletonem.

Domyślnie mamy dwa zakresy: **singleton** i **prototype**, w webowym - MVC kontekście dochodzi jeszcze **request** (obiekty które są tworzone dla żądania i po zakończeniu przetwarzania żądania te obiekty są automatycznie niszczone) i **session** (sesja użytkownika po stronie serwera jest utworzona i obiekty sesji są skojarzone z sesją i dopiero walidacja sesji spowoduje wyrzucenie obiektów z pamięci i zniszczenie).
Singleton oznacza jeden obiekt utworzony dla tej definicji i za każdym razem, gdy klient odpytuje klasę, aby otrzymać beana o tej definicji otrzymuje **tą samą instancję**.
Prototyp oznacza, że za każdym razem kiedy będzie do kontekstu springowego request o wyciagnięcie takiego beana to zostanie dostarczona **nowa instancja**.

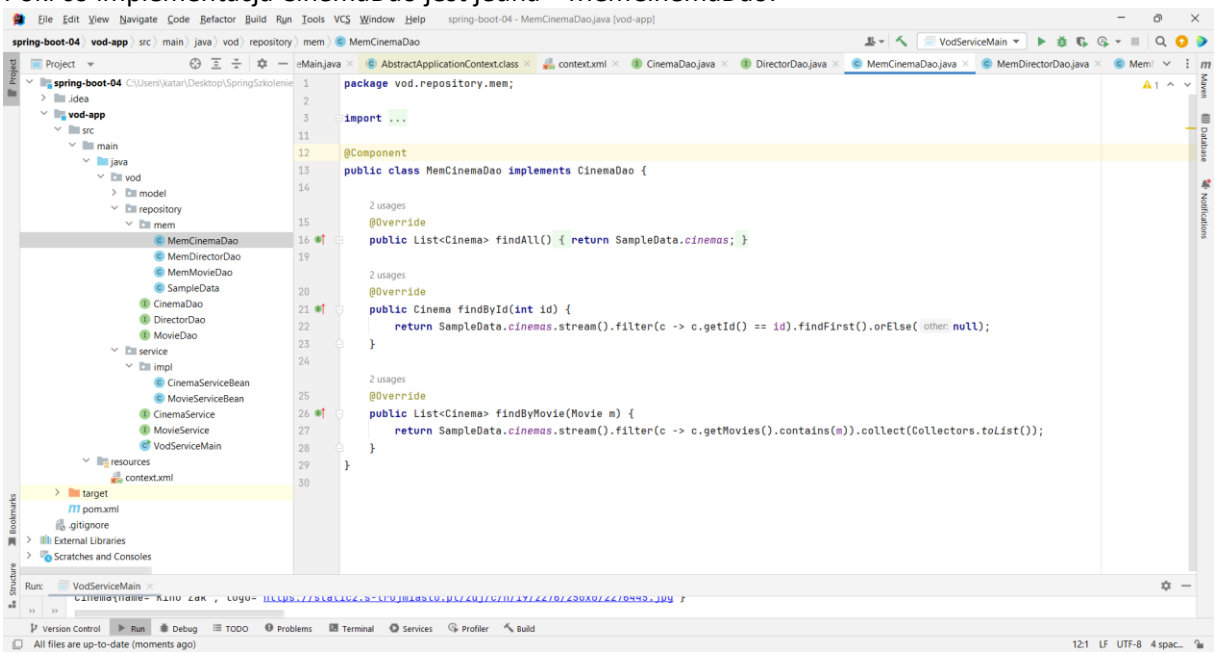


Tym razem beany service i service2 mają już inną referencję, ale w środku składowe mają takie same. Czyli **scope nie propaguje się kaskadowo**.

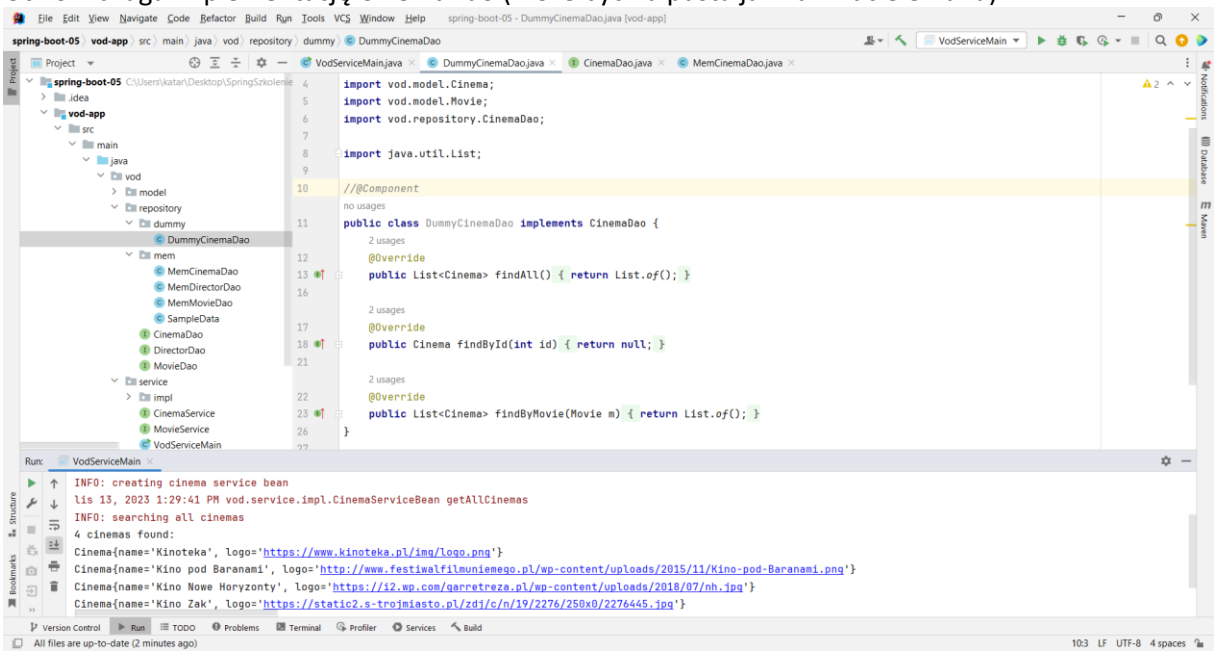
Repozytoria powinny być bezstanowe i to czy korzystamy z jednej czy drugiej instancji powinno być przeźroczyste.
Dzięki scope można zarządzać krotnościami instancji.

Cykl życia beana jest zarządzany przez Springa poprzez ustawienia scope.

W Spring Boot nie będziemy widzieli ani kontekstu ani wyciągali beana metodą getBean tylko będziemy deklarowali w kontekście naszej klasy, że ma się on automatycznie pojawić.
Na początek poznajemy fundamenty kontenera springa – umieszczanie obiektów, cykl życia, rozróżniania implementacji gdyby ich było kilka.
Póki co implementacja CinemaDao jest jedna – MemCinemaDao.



Utwórz druga implementację CinemaDao (może być na pusta jak na zrzucie ekranu)



Czy umieszczenie drugiej implementacji coś zmieni?
Uruchom program.

Jeśli tylko jedna jest oznaczona adnotacją Component to tylko ona będzie utworzona i umieszczona w kontekście Springa.

```
import ...

11 usages   2 implementations
public interface CinemaDao {

    Choose Implementation of CinemaDao (2 found)
    DummyCinemaDao (vod.repository.dummy)   vod-app
    MemCinemaDao (vod.repository.mem)       vod-app

    2 usages   2 implementations
    Cinema findById(int id);

    2 usages   2 implementations
    List<Cinema> findByMovie(Movie m);

}
```

Widzimy, że interfejs ma dwie implementacje.
Rozwikływanie zależności-wiązanie dzieje się w czasie wykonania w runtimie nie na etapie kompilacyjnym, dopiero na etapie podnoszenia kontekstu springa, przeskanowania klas i zbudowanie drzewa zależności beanów na starcie aplikacji w momencie podnoszenia kontekstu sprinowego.

Druga implementacja nie została umieszczona w kontekście bo nie ma adnotacji, dodaj ją:

```
dummy
    DummyCinemaDao
mem
    MemCinemaDao
    MemDirectorDao
    MemMovieDao
    SampleData
CinemaDao
DirectorDao
MovieDao
service
impl
    CinemaServiceBean
    MovieServiceBean
CinemaService
MovieService
VodServiceMain
```

```
import vod.repository.CinemaDao;

import java.util.List;

@Component
public class DummyCinemaDao implements CinemaDao {
    2 usages
    @Override
    public List<Cinema> findAll() { return List.of();

    2 usages
    @Override
    public Cinema findById(int id) { return null; }

    2 usages
    @Override
    public List<Cinema> findByMovie(Movie m) { return
```

Uruchom.

```
package vod.repository.mem;

import ...

//@Component("cinemaDao")
@Component
public class MemCinemaDao implements CinemaDao {

    2 usages
    @Override
```

```
Run: VodServiceMain
    at org.springframework.beans.factory.support.AbstractBeanFactory.lambda$doGetBean$0(AbstractBeanFactory.java:335)
    at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton(DefaultSingletonBeanRegistry.java:234)
    at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:333)
    at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:208)
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.preInstantiateSingletons(DefaultListableBeanFactory.java:955)
    at org.springframework.context.support.AbstractApplicationContext.finishBeanFactoryInitialization(AbstractApplicationContext.java:921)
    at org.springframework.context.support.AbstractApplicationContext.refresh(AbstractApplicationContext.java:583)
    at org.springframework.context.annotation.AnnotationConfigApplicationContext.<init>(AnnotationConfigApplicationContext.java:105)
    at vod.service.VodServiceMain.main(VodServiceMain.java:20)
Caused by: org.springframework.beans.factory.NoUniqueBeanDefinitionException: No qualifying bean of type 'vod.repository.CinemaDao' available: expected single match
    at org.springframework.beans.factory.config.DependencyDescriptor.resolveNotUnique(DependencyDescriptor.java:220)
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.doResolveDependency(DefaultListableBeanFactory.java:1369)
    at org.springframework.beans.factory.support.DefaultListableBeanFactory.resolveDependency(DefaultListableBeanFactory.java:1311)
    at org.springframework.beans.factory.support.ConstructorResolver.resolveAutowiredArgument(ConstructorResolver.java:911)
    at org.springframework.beans.factory.support.ConstructorResolver.createArgumentArray(ConstructorResolver.java:788)
    ... 14 more

Process finished with exit code 1
```

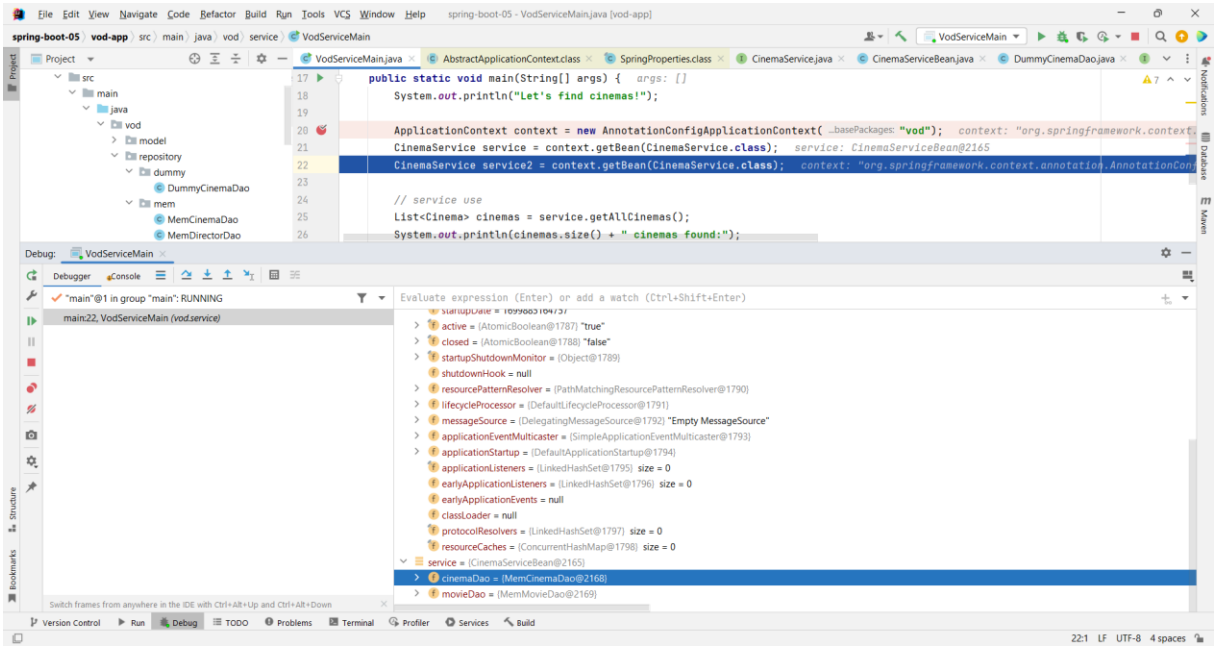
Mamy wyjątek :
Caused by: org.springframework.beans.factory.NoUniqueBeanDefinitionException: No qualifying bean of type 'vod.repository.CinemaDao' available: expected single matching bean but found 2: dummyCinemaDao,memCinemaDao

To są identyfikatory wzięte z nazwy klas, utworzone domyślnie przez Springa.
Znalazł dwie implementacje cinemaDao i nie może ich obu dostarczyć do serwisu, bo ma dwóch kandydatów do wstrzyknięcia i nie wie którego użyć.

Trzeba wskazać, która implementacja ma być użyta, bo tym razem Spring utworzył obie i obie umieścił w kontekście.
Można zadziałać w kilka strategii – 1. Dodaj adnotację **Primary**

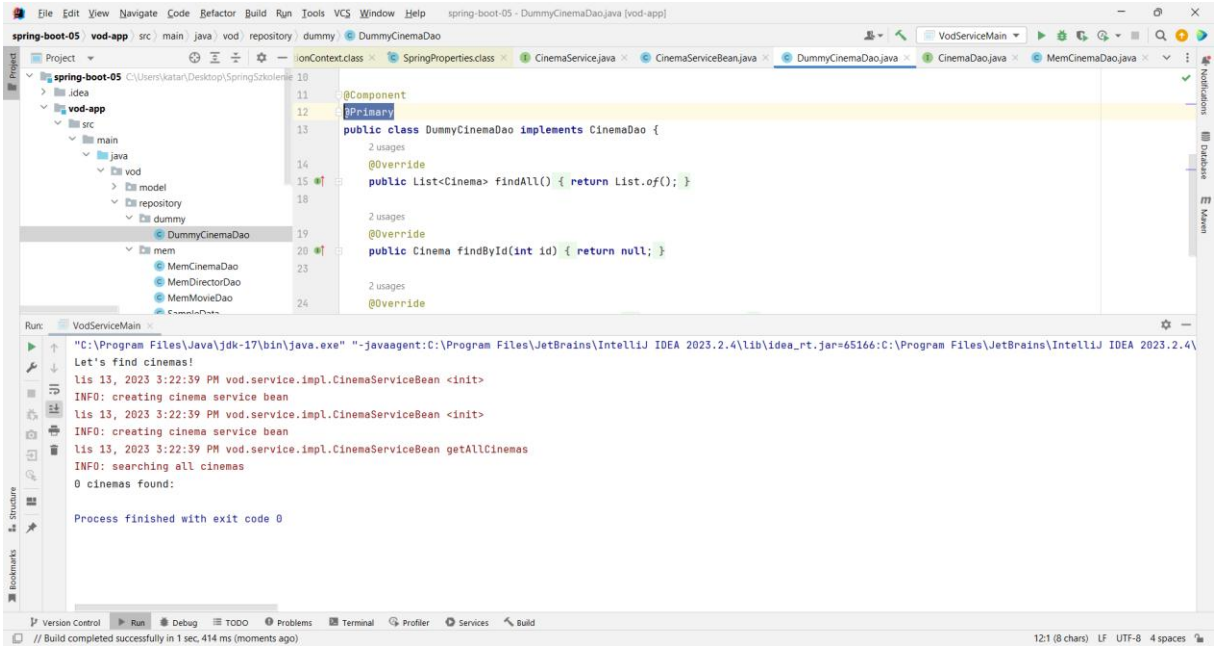
```
import org.springframework.stereotype.Component;
import vod.model.Cinema;
import vod.model.Movie;
import vod.repository.CinemaDao;

import java.util.List;
import java.util.stream.Collectors;

//@Component("cinemaDao")
@Component
@Primary
public class MemCinemaDao implements CinemaDao {

    2 usages
```

```
Run: VodServiceMain
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2023.2.4\lib\idea_rt.jar=64005:C:\Program Files\JetBrains\IntelliJ IDEA 2023.2.4\
Let's find cinemas!
lis 13, 2023 2:00:24 PM vod.service.impl.CinemaServiceBean <init>
INFO: creating cinema service bean
lis 13, 2023 2:00:24 PM vod.service.impl.CinemaServiceBean <init>
INFO: creating cinema service bean
lis 13, 2023 2:00:24 PM vod.service.impl.CinemaServiceBean getAllCinemas
INFO: searching all cinemas
4 cinemas found:
Cinema{name='Kinoteka', logo='https://www.kinoteka.pl/img/logo.png'}
Cinema{name='Kino pod Baranami', logo='http://www.festiwalfilmuniemego.pl/wp-content/uploads/2015/11/Kino-pod-Baranami.png'}
Cinema{name='Kino Nowe Horyzonty', logo='https://i2.wp.com/garretreza.pl/wp-content/uploads/2018/07/nh.jpg'}
Cinema{name='Kino Zak', logo='https://static2.s-trojmiasto.pl/zdj/c/n/19/2276/250x0/2276445.jpg'}

Process finished with exit code 0
```



Jeżeli ustawimy Primary nad implementacja DummyCinemaDao otrzymamy pustą listę kin zgodnie z implementacją.

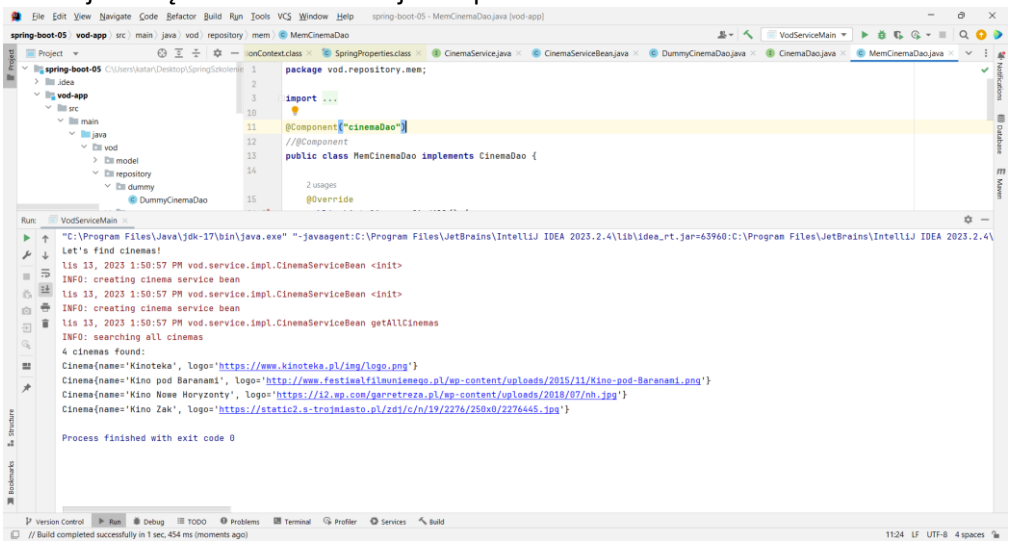
To strategia od strony dostarczyciela Beana.



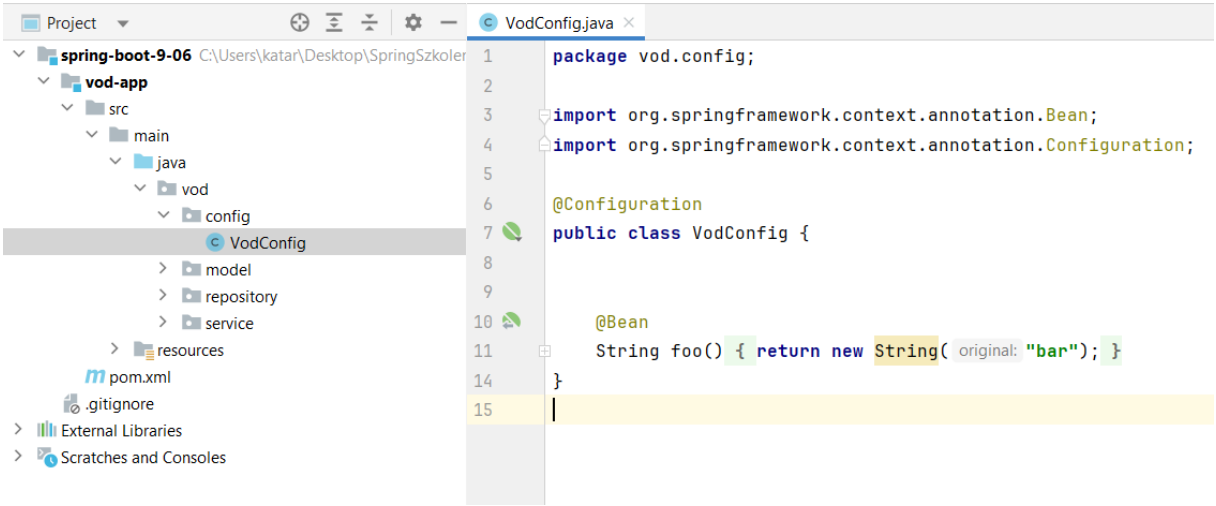
Strategia 2 – od strony użycia
Niejednoznaczność mamy w tym miejscu:

```
private MovieDao movieDao;

//@Autowired
public CinemaServiceBean(CinemaDao cinemaDao, MovieDao movieDao) {
    log.info( msg: "creating cinema service bean");
    this.cinemaDao = cinemaDao;
    this.movieDao = movieDao;
}
```

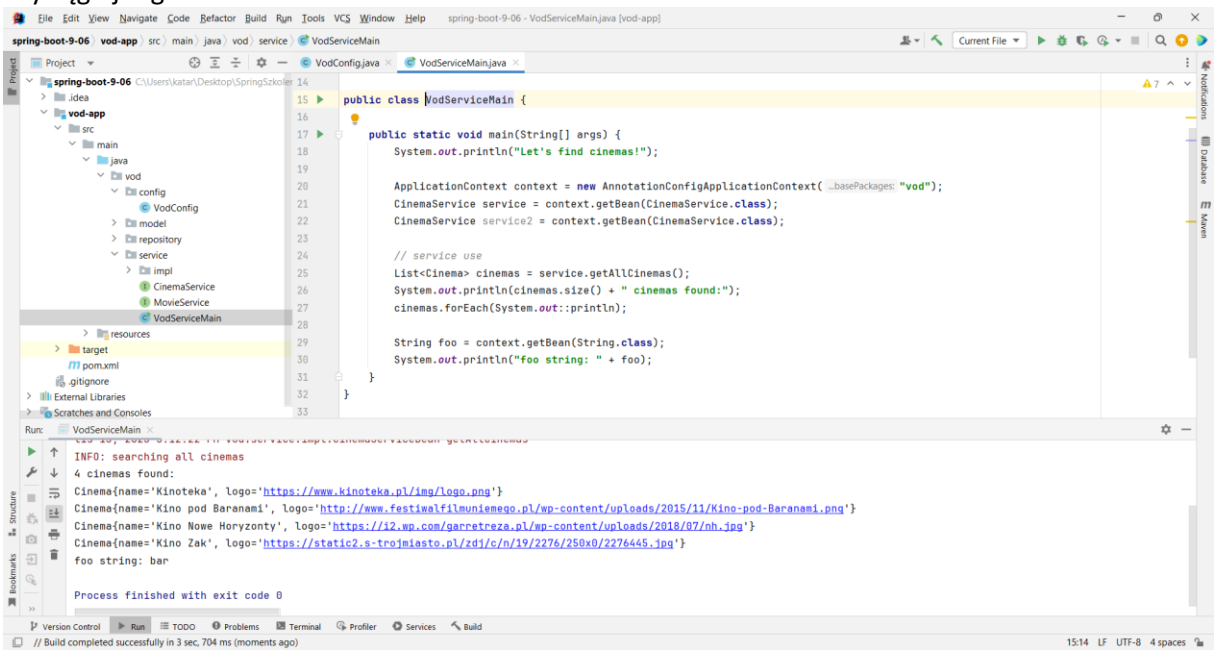
- dodaj nazwę beana do adnotacji Component i uruchom.



W pakiecie vod utwórz **pakiet config** oraz klasę konfiguracyjną.
Oznacz ją adnotacją **Configuration** – informujemy Springa, że w niej jest cos ważnego i należy ją wykonać.
Adnotację Component mogę wykonać na kodzie którego jestem właścicielem (typu), ale jeśli chcemy do kontekstu springowego wprowadzić klasę, która nie jest nasza tylko np. ze standardowego API Javy np. typu String to możemy stworzyć metodę opisaną **adnotacją Bean**.
Spring przy bootstrapie przy starcie po odnalezieniu klasy konfiguracyjnej i metody oznaczonej adnotacją Bean wywoła tą metodę i to zostanie zwrócone i umieszczone w kontekście springowym pod identyfikatorem takim jak nazwa metody – tutaj foo



Obiekt ten będzie utworzony przy starcie aplikacji i umieszczony w kontekście springowym pod identyfikatorem foo.
Nie ma to znaczenia czy obiekt będzie utworzony poprzez skanowanie klas czy poprzez wykonanie metod oznaczonych bean z konfiguracji.
Czyli klasa konfiguracyjna pozwala na umieszczenie kolejnego obiektu w kontekście springowym.
Wyciągnij tego beana w mainie.



W ten sposób dowolne obiekty możemy jawnie imperatywnie tworzyć, nadawać im jakiś stan i zwracać je do kontekstu Springa.